NLP CS60075 Assignment II — Datasets

How to read this document. Every dataset used anywhere across the 11 projects has one entry here: where to get it, its exact file/field format, its official splits (or how to construct one if it has none), any preprocessing you must do before using it, and – separately for every task that uses it – exactly how it is meant to be used in that task. A dataset reused by more than one project (e.g. UD English EWT, used in Projects 2, 3, and 5) is documented in full only once, at its first appearance; every later reuse is a short entry that points back to that first entry and states only what is genuinely different about how the later project uses the same file. Section numbers below match the "companion Datasets document, Section N" references already given in the Projects document.

1 Project 1 Datasets: N-gram LM, Smoothing, and Neural LM Reproduction

Datasets 1.1–1.3 below are the three *assignable* datasets – your group works with whichever one you are assigned, using it for all three tasks. Dataset 1.B (BLiMP) is a separate, shared bonus check that every group runs identically, regardless of which of 1.1–1.3 they were assigned.

Dataset 1.1 — WikiText-2

Source

`https://huggingface.co/datasets/Salesforce/wikitext`, configuration `wikitext-2-raw-v1`. Load with `load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")`. License: CC-BY-SA-3.0.

Format

A single text column. Each example is one line of raw Wikipedia article text as it appears in the article, including section-heading lines (wrapped in `= =`) and blank lines separating articles.

Field	Type	Meaning
`text`	string	One line of raw article text, a heading line, or an empty string (article separator).

Splits

Split	Size	Notes
`train`	≈2.09M tokens	Use for all training in Tasks 1–3.
`validation`	≈218K tokens	Hyperparameter tuning only (k, λ, checkpoint selection). Never touch this for final numbers.
`test`	≈246K tokens	Final reported perplexity only.

Required Preprocessing

1. Concatenate all `text` lines within a split into one long token stream – WikiText's lines are a storage artifact, not sentence boundaries.
2. Tokenize with a punctuation-aware whitespace tokenizer and lowercase every token.
3. Build the vocabulary from `train` only: keep the 30,000 most frequent tokens; map every other token to `<UNK>`. Tokens occurring < 2 times in train also map to `<UNK>`.

Task 1 use: If assigned this dataset: train n-gram count tables on `train`; tune smoothing hyperparameters on `validation`; report final perplexity on `test`.

Task 2 use: If assigned this dataset: train the AWD-LSTM on `train`; use `validation` for checkpoint selection if deviating from a fixed epoch count; report final perplexity on `test`.

Task 3 use: If assigned this dataset: no additional training – reuse the Task 1/2 models. Use `validation` only to tune λ; report final perplexity on `test`.

Dataset 1.2 — DailyDialog

Source

`https://huggingface.co/datasets/daily_dialog`. Load with `load_dataset("daily_dialog")`.

Format

Each example is one multi-turn dialogue.

one-dataset-per-group design (see the project's Problem Statement).

Task 1 use: If assigned this dataset: build the word-word co-occurrence matrix and PPMI+SVD vectors from this dataset's train-split sentences.

Task 2 use: If assigned this dataset: train/evaluate the structural probe on `train`/`test`.

Task 3 use: If assigned this dataset: extract BERT-derived static vectors from this dataset's train-split sentences.

Dataset 5.B (Bonus, shared across every group) — WordSim-353 and the Google Analogy Set

Source

WordSim-353: `https://huggingface.co/datasets/enus5three/WordSim353`. Google analogy set: distributed alongside the original word2vec release, `https://github.com/tmikolov/word2vec` (file `questions-words.txt`).

Format

Two files bundled as one evaluation suite: WordSim-353 is word pairs with a human similarity judgment; the analogy set is 4-word analogy questions.

Field	Type	Meaning
`word1`, `word2`	string	The two words being judged (WordSim-353).
`similarity`	float	Human-judged similarity, 0–10 scale (WordSim-353).
line format `a b c d`	4 strings	One analogy question: "a is to b as c is to ?", gold answer d.

Splits

No split – both are pure evaluation sets (353 pairs, ≈19,544 questions), used in full by every group regardless of which of Datasets 5.1–5.3 they were assigned.

Required Preprocessing

1. Lowercase every word to match the vocabulary casing from your assigned dataset.
2. Drop any analogy question with an out-of-vocabulary word; report how many were dropped, since your vocabulary size depends on which treebank you were assigned.
3. Compute Spearman (not Pearson) correlation for WordSim-353 – the task only needs relative ordering, not the human 0–10 scale matched numerically.

Task 1 use: Compute WordSim-353 Spearman correlation and Google-analogy accuracy for the PPMI+SVD vectors.

Task 2 use: Not used (Task 2 is evaluated with UUAS/depth-correlation instead, a structural rather than similarity/analogy metric).

Task 3 use: Compute the identical two metrics for the BERT-derived vectors, same protocol as Task 1.

6 Project 6 Datasets: A Text Processing Pipeline for Code-Mixed, Real-World Text

Datasets 6.1–6.3 below are three disjoint partitions of the same underlying SemEval SentiMix corpus, so every group gets a genuinely self-contained, correctly-labeled Hindi-English code-mixed sentiment dataset. Dataset 6.B (Dakshina) is a separate, shared resource every group uses identically for normalization.

Dataset 6.1 — SemEval SentiMix, Partition A

Source

Three files from one repository, all at `https://github.com/singhnivedita/SemEval2020-Task9`:

- `Train.txt`, `Validation.txt`, `Test.txt` in `https://github.com/singhnivedita/SemEval2020-Task9/tree/master/Pre%20Processing%20Code/Original%20Dataset` – the official SemEval-2020 Task 9 Hinglish tweets, tokenized with per-token language tags. `Test.txt` carries *no* sentiment labels.
- `Ty.txt` in `https://github.com/singhnivedita/SemEval2020-Task9/blob/master/Fully%20Processed%20Datasets/Ty.txt` – the gold sentiment labels for `Test.txt` (CSV with header `Uid,Sentiment`).

Download the files directly – no account, gating, or task-page dependency. **Do not use** the `festvox/sentimix_hi_en` HuggingFace mirror (access error) or the `ritual-uh.github.io/sentimix2020` task page (unreliable from some networks). **Do not use** the stemmed `.tsv` files elsewhere in the repository: they have no language tags, so Task 1's language-ID step cannot run on them.

Format

Each example is one code-mixed tweet in tokenized CoNLL-style format: a header line `meta<TAB><id><TAB><sentiment>`, followed by one line per token (`token<TAB>language_tag`), followed by a blank line separating tweets. In `Test.txt` the header has no sentiment field; take it from `Ty.txt` by id. Combine all three files into one pool of ≈18,480 labelled tweets before partitioning (see Required Preprocessing) – the original train/validation/test division is not the one used here, since every group needs its own complete, self-contained partition.

Field	Type	Meaning
tweet header	string	One line per tweet: `meta`, the tweet's numeric id, and its sentiment (`positive`, `negative`, or `neutral`).
token	string	One word/token of the tweet, one per line following its header.
lang_tag	string	Per-token language tag: `Hin`, `Eng`, or `O` (other – punctuation, symbols, emojis, URL fragments).
Uid, Sentiment	string	(`Ty.txt` only) test-tweet id and its gold sentiment label.

Splits

Split	Size	Notes
Partition A / train	≈4,928 tweets	Roughly 80% of this partition's third (6,160 tweets) of the combined pool.
Partition A / validation	≈616 tweets	Roughly 10%.
Partition A / test	≈616 tweets	Roughly 10%.

Required Preprocessing

1. Parse each file by blank-line-separated blocks. A header line is one that matches `meta<TAB><id><TAB><sentiment>` exactly; do *not* treat every line beginning with "meta" as a header, since ordinary tokens such as `metaphysist` also begin that way. All other lines, each `token<TAB>lang_tag`, are that tweet's tokens.
2. Read `Ty.txt` and attach its sentiment to each `Test.txt` tweet by id.
3. **Remove duplicate tweets.** About 2,650 tweet ids appear in both `Train.txt` and `Validation.txt` (same text, same label). Keep one copy per id; otherwise the same tweet can land in different groups' train and test splits. The deduplicated pool is ≈18,480 tweets.
4. **Construct all three partitions with one shared procedure** (do this once; use the identical seed across the whole class): shuffle the pool once with a fixed seed, cut into three equal thirds – Partition A, B, C – then split each third 80/10/10 into its own train/validation/test.
5. Join tokens with single spaces for the TF-IDF/MuRIL pipelines, but keep the original token list (and aligned `lang_tag` list) available separately – Task 1's language-ID classifier is trained per token.
6. Raw tweet text here is messy in realistic ways: URLs are frequently split across several tokens (e.g. `https`, `//`, `t`, `.`, `co`, `/`, `<id>`), usernames are tagged inconsistently depending on script, and emojis are tagged `O`. Decide on and document one consistent handling (e.g. strip URL-fragment tokens before building TF-IDF features) rather than leaving it implicit.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest sentiment classifier on Partition A's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition A's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition A's train split; evaluate macro-F1 on its test split.

Dataset 6.2 — SemEval SentiMix, Partition B

Source, Format, Splits, Preprocessing

Identical to Dataset 6.1's construction procedure (Section 6) – not repeated here. The second third of the same shuffled pool (≈6,160 tweets), further divided 80/10/10 into its own train/validation/test.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest on Partition B's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition B's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition B's train split; evaluate macro-F1 on its test split.

Dataset 6.3 — SemEval SentiMix, Partition C

Source, Format, Splits, Preprocessing

Identical to Dataset 6.1's construction procedure (Section 6) – not repeated here. The remaining third of the same shuffled pool (≈6,160 tweets), further divided 80/10/10 into its own train/validation/test.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest on Partition C's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition C's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition C's train split; evaluate macro-F1 on its test split.

Dataset 6.B (Shared resource, used identically by every group) — Dakshina (Hindi Transliteration Pairs)

Source

`https://github.com/google-research-datasets/dakshina`. Use only the `hi` (Hindi) portion's word-pair romanization lexicon (not its separate full-sentence transliteration data, which is not used here).

Format

Each entry pairs a native-script (Devanagari) Hindi word with one or more attested Romanized spellings, each with a frequency count.

Field	Type	Meaning
native word	string	The word in Devanagari script.
romanization(s)	string(s)	One or more attested Romanized spellings of that word.
count	int	How many times each romanization was observed.

Splits

Use the full word-pair lexicon (all of Dakshina's own splits combined) – this is a fixed, static reference dictionary, identical for every group regardless of which SemEval partition (6.1, 6.2, or 6.3) they were assigned.

Required Preprocessing

1. Build a Romanized-spelling → canonical-form lookup dictionary: for each native word, treat its most frequent romanization as canonical, and map every other attested romanization of that word to it.
2. This dictionary is a static rule-based lookup, never trained or fit to any group's tweet partition, so there is no risk of leaking test labels into it.

Task 1 use: Build the normalization dictionary from this lexicon; apply it to your assigned partition's text before TF-IDF feature extraction.

Task 2 use: Apply the same dictionary to your assigned partition's test text at evaluation time only (the additional check described in Project 6's Task 2), never during MuRIL's raw-text training pass.

Task 3 use: Apply the same dictionary as part of the full preprocessing chain, alongside the LID-augmentation step.

7 Project 7 Datasets: Do LLMs Know When They're Making Things Up?

Dataset 7.1 — WikiBio GPT-3 Hallucination Dataset

Source

`https://huggingface.co/datasets/potsawee/wiki_bio_gpt3_hallucination` (from the SelfCheckGPT paper, Manakul et al., 2023).

Format

Each example is one GPT-3-generated, Wikipedia-style biography passage, split into sentences, with a human accuracy annotation per sentence, plus 20 independently resampled passages from the same prompt.